

# LAB MANUAL

**Course: CSC-241**



**Department of Computer Science**

**COMSATS UNIVERSITY ISLAMABAD**

Lahore Campus

# Table of Content

| Lab #    | Topics Covered                                                             | Page # |
|----------|----------------------------------------------------------------------------|--------|
| Lab # 01 | Difference between Object Oriented and Procedural Programming:             | 03     |
|          | Installation of JDK, Basic Syntax of Java Program                          |        |
| Lab # 02 | Primitive Data Types, Ranges and their literals in Java                    | 09     |
|          | Conditions and Loops in Java                                               |        |
| Lab # 03 | Defining classes, objects and Methods in JAVA                              | 11     |
|          | User Input in Java                                                         |        |
| Lab # 04 | Controlling access to class members – Encapsulation                        | 17     |
|          | Defining classes and objects in JAVA through constructors                  |        |
| Lab # 05 | Static Data members and Methods                                            | 22     |
|          |                                                                            |        |
|          | <b>Lab Sessional-I</b>                                                     |        |
| Lab # 06 | Composition / Containership(Has-a relationship)                            | 28     |
|          |                                                                            |        |
| Lab # 07 | One dimensional & Multi-dimensional primitive and reference arrays in java | 37     |
|          |                                                                            |        |
| Lab # 08 | ArrayList in Java                                                          | 42     |
|          |                                                                            |        |
| Lab # 09 | Inheritance, Method Overloading, Polymorphism                              | 46     |
|          |                                                                            |        |
| Lab # 10 | Abstarction                                                                | 59     |
|          |                                                                            |        |
|          | <b>Lab Sessional-II</b>                                                    |        |
| Lab # 11 | Interfaces                                                                 | 63     |
|          |                                                                            |        |
| Lab # 12 | Exception Handling                                                         | 74     |
|          |                                                                            |        |
| Lab # 13 | Database Connectivity                                                      | 82     |
|          |                                                                            |        |
|          | <b>Terminal Examination</b>                                                |        |

### **Statement Purpose:**

Objective of this lab is to make students understand the basic syntax of java specifically conditions and loops

### **Activity Outcomes: (CLO4-PL05/P1)**

After this lab, students will be able to install Java 1.8 with Notepad++ editor or IDE and also they would learn the basic syntax of java.

### **Instructor Note:**

The Students should have knowledge about structured programming

## Introduction

Microprocessor are key component in consumer-electronics 1991 - Sun Microsystems funded an internal project led by James Gosling, to write a language that shall execute on all microprocessor for consumer-electronics. In 1993, the web wide adaptation given rise to Java because Sun Microsystems launched features to create web applications. Java become popular because of web, not for its initial goal. In 2009, Oracle acquired Sun Microsystems.

Some features of Java are given below:

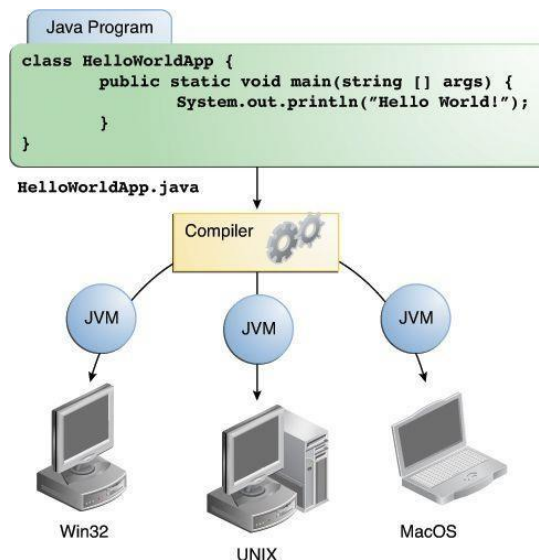
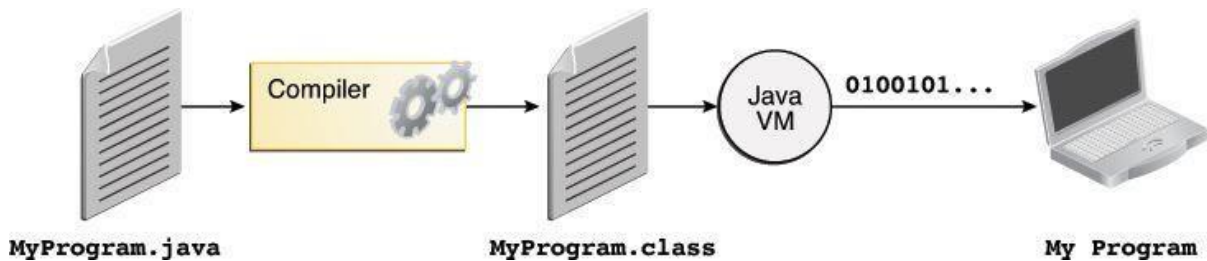
Portability: Write Once, Run Anywhere

Robust, Secure and Reliable – No pointer arithmetic, compile time error checking, bytecode verification, etc.

Strong libraries of components available

Supports distributed applications, multithreading

Life cycle of java program is given below

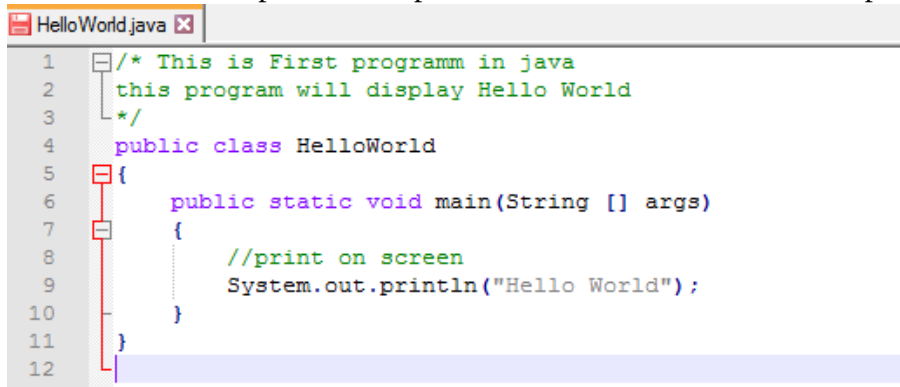


## Lab Activities

## Activity 1

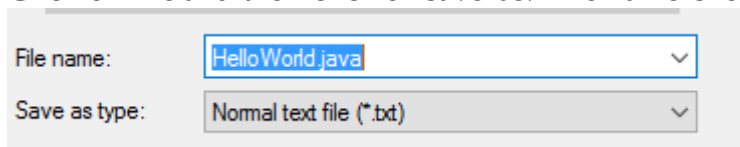
This example will demonstrate use of notepad++ for executing simple java program.

Write code in notepad or notepad++. Write the code which displays “Hello World”.



```
1  /* This is First programm in java
2     this program will display Hello World
3  */
4  public class HelloWorld
5  {
6      public static void main(String [] args)
7      {
8          //print on screen
9          System.out.println("Hello World");
10     }
11 }
12
```

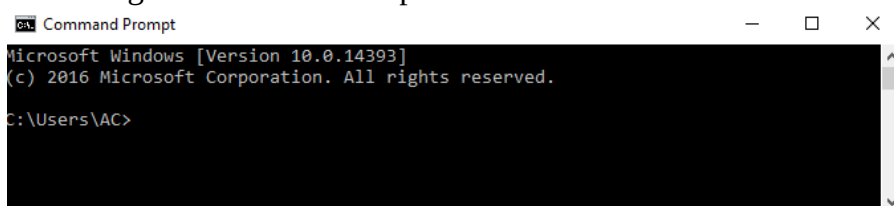
Click on file and then click on save as. File name should be same as class name.



Create a folder in C drive. Save the program in new created folder.

Now open the command prompt from All Programs -> Accessories -> command prompt.

Following window will be opened.



Write following commands on command prompt.

cd\ (change directory)

cd FolderName

**To Compile:**

javac HelloWorld.java

'javac' is not recognized as an internal or external command,  
operable program or batch file.

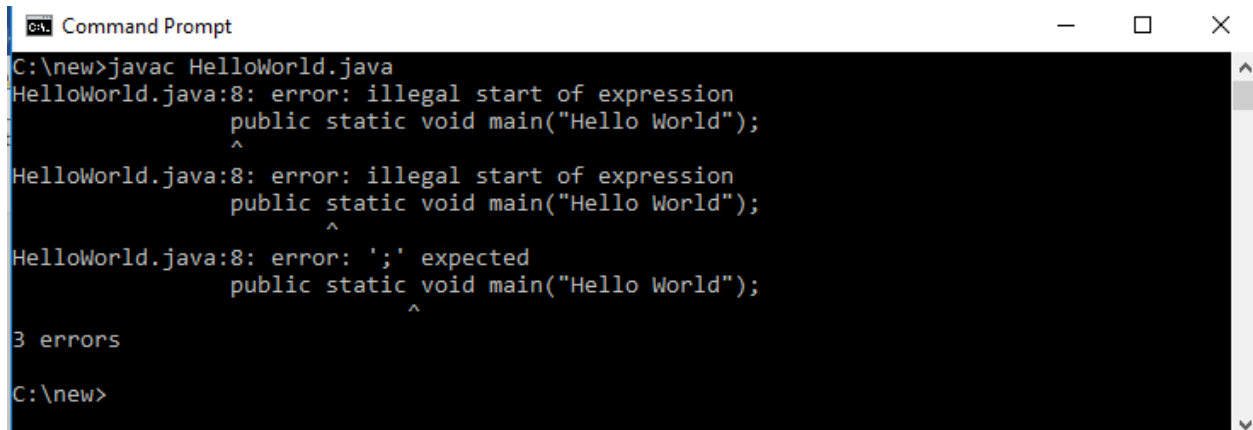
C -> program files -> java -> jdk version -> bin -> right click -> properties -> copy path  
C:\Program Files\Java\jdk1.8.0\_161\bin

**TEMPORARY PATH SETTING**

Command prompt -> set path= C:\Program Files\Java\jdk1.8.0\_161\bin

Javac HelloWorld.java

If there is no syntax error, then file will be compiled successfully and a new file with same name will be created in same directory with extension “.class”. You can verify it by checking your folder yourself.



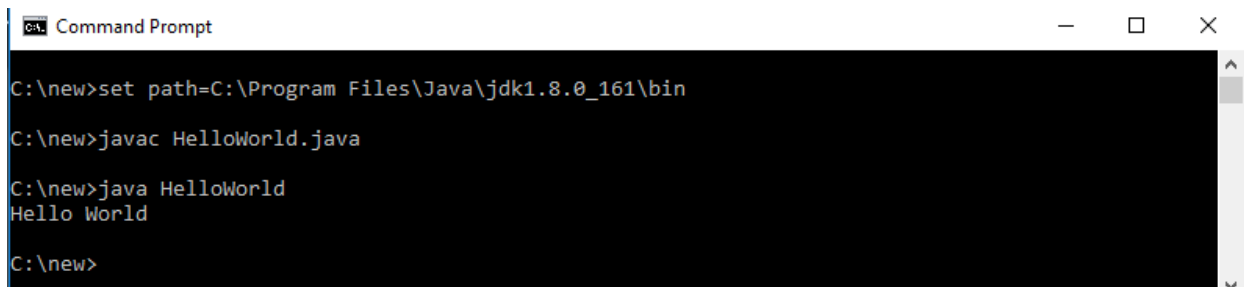
```
Command Prompt
C:\new>javac HelloWorld.java
HelloWorld.java:8: error: illegal start of expression
        public static void main("Hello World");
        ^
HelloWorld.java:8: error: illegal start of expression
        public static void main("Hello World");
        ^
HelloWorld.java:8: error: ';' expected
        public static void main("Hello World");
                               ^
3 errors
C:\new>
```

Write now it is showing errors. I will correct the error and recompile the file after saving it. Error is rectified. In addition, compiler has not shown any error.

### To Run:

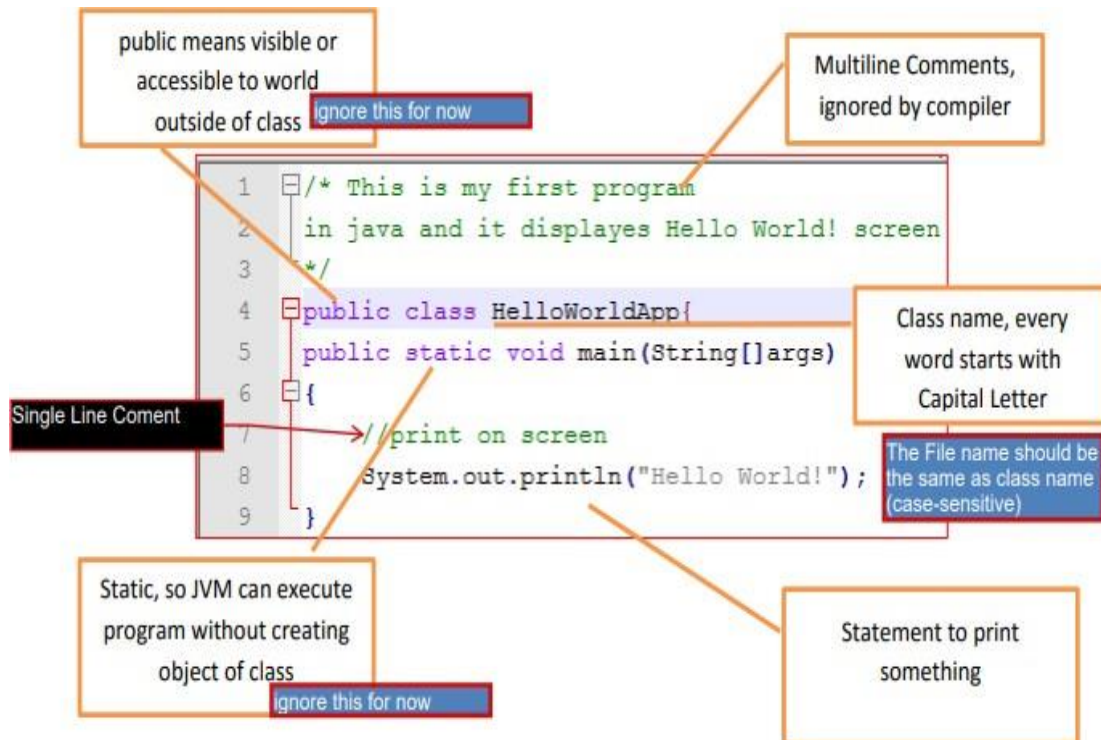
java HelloWorld

Now write java FileName and press enter, if there is no logical error then you will see the output “HelloWorld!” at screen.



```
Command Prompt
C:\new>set path=C:\Program Files\Java\jdk1.8.0_161\bin
C:\new>javac HelloWorld.java
C:\new>java HelloWorld
Hello World
C:\new>
```

### Explanation of Program

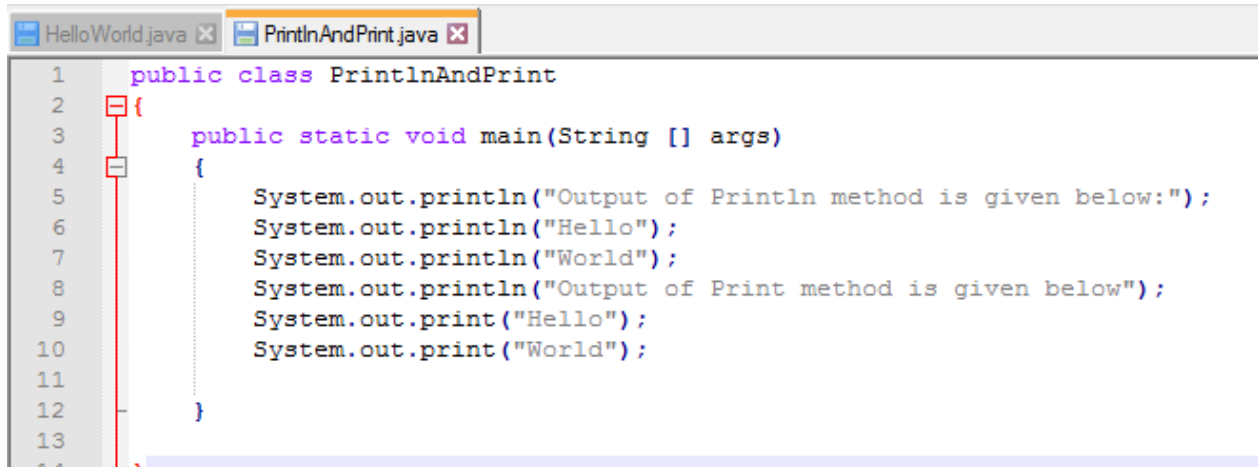


### Common Errors

1. If name of file and name of class is different
2. If main method parenthesis is empty or wrong parameter are given.
3. If any matching bracket is missing
4. Semicolon is missing
5. If main method is private and not public
6. If class is private and not public
7. If main method is not static

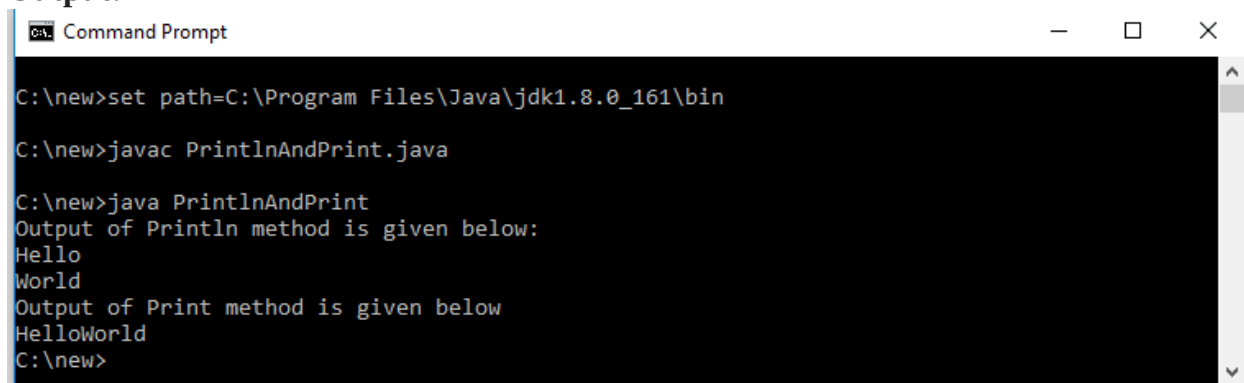
### Print vs Println

The `print("aString")` method **prints** just the string "aString", but does not move the cursor to a new line. Hence, subsequent **printing** instructions will **print** on the same line. The `println("aString")` method **prints** the string "aString" and moves the cursor to a new line.



```
1 public class PrintlnAndPrint
2 {
3     public static void main(String [] args)
4     {
5         System.out.println("Output of Println method is given below:");
6         System.out.println("Hello");
7         System.out.println("World");
8         System.out.println("Output of Print method is given below");
9         System.out.print("Hello");
10        System.out.print("World");
11    }
12
13
14
```

### Output:



```
C:\new>set path=C:\Program Files\Java\jdk1.8.0_161\bin
C:\new>javac PrintlnAndPrint.java
C:\new>java PrintlnAndPrint
Output of Println method is given below:
Hello
World
Output of Print method is given below
HelloWorld
C:\new>
```

## Home Activities

### Activity 1:

Install Java 1.8 with editor Notepad++/Notepad or any IDE. Once you are done with installation write the code to print “Hello World” in Java.

### **Statement Purpose:**

Objective of this lab is to make students understand primitive data types in java and how to use conditions and loops.

### **Activity Outcomes: (CLO4-PO5/P1)**

After this lab, students will be able to use primitive data types in Java.  
Students will learn syntax of conditions and loops in Java.

### **Instructor Note:**

The Students should have knowledge about data types and structure of loops and conditions in structured programming.

# **Introduction**

## **Home Activities**

### **Activity 1:**

Write a java code, which prints student's information i.e. Roll number, marks in assignment1, GPA, CGPA, gender (print M for male and F for female), hostelite (True/False) or not. For each information, you have to use appropriate data type.

### **Activity 2:**

Write java code that prints table of 5.

### **Activity 3:**

Write java code that prints even and odd numbers from 1 to 10.

### **Activity 4:**

Write a code, to check whether a given number is positive number or negative number.

### **Activity 5:**

Write java code, which prints grades of students according to marks obtained.

If marks are greater than or equal to 90 print A grade

If marks are greater than or equal to 80 and less than 90 print B grade

If marks are greater than or equal to 70 and less than 80 print C grade

If marks are greater than or equal to 60 and less than 70 print D grade

If marks are greater than or equal to 50 and less than 60 print E grade

If marks are less than 50 print F grade.

### **Activity 6:**

Write java code, which prints sum of first ten natural numbers.

### **Activity 7:**

Write java code, which prints factorial of given number.

### **Activity 8:**

Write java code, which prints the number in reverse order.

**Expected Output:** Number 1234

### **Statement Purpose:**

Objective of this lab is to make students understand the difference between object oriented and procedural approaches to programming

### **Activity Outcomes: (CLO4-PL05/P2)**

The student will understand the advantages of using OOP

The student will be able to understand difference between class and object.

### **Instructor Note:**

The students should brainstorm about the scenarios given in activities; in order to model them in terms of Objects.

## **Introduction**

Procedural programming uses a list of instructions to tell the computer what to do step-by-step. Procedural programming relies on procedures, also known as routines or subroutines. A procedure contains a series of computational steps to be carried out.

Procedural programming is intuitive in the sense that it is very similar to how you would expect a program to work. If you want a computer to do something, you should provide step-by-step instructions on how to do it. It is, therefore, no surprise that most of the early programming languages are all procedural. Examples of procedural languages include Fortran, COBOL and C, which have been around since the 1960s and 70s.

Object-oriented programming, or OOP, is an approach to problem-solving where all computations are carried out using objects. An object is a component of a program that knows how to perform certain actions and how to interact with other elements of the program. Objects are the basic units of object-oriented programming. A simple example of an object would be a person. Logically, you would expect a person to have a name. This would be considered a property of the person. You would also expect a person to be able to do something, such as walking. This would be considered a method of the person.

A method in object-oriented programming is like a procedure in procedural programming. The key difference here is that the method is part of an object. In object-oriented programming, you organize your code by creating objects, and then you can give those objects properties and you can make them do certain things.

One of the most important characteristics of procedural programming is that it relies on procedures that operate on data - these are two separate concepts. In object-oriented programming, these two concepts are bundled into objects. This makes it possible to create more complicated behavior with less code. The use of objects also makes it possible to reuse code. Once you have created an object with more complex behavior, you can use it anywhere in your code.

The world around us is made up of objects, such as people, automobiles, buildings, streets, and so forth. Each of these objects has the ability to perform certain actions, and each of these actions has some effect on some of the other objects in the world.

OOP is a programming methodology that views a program as similarly consisting of objects that interact with each other by means of actions.

Object-oriented programming has its own specialized terminology. The objects are called, appropriately enough, objects. The actions that an object can take are called methods. Objects of the same kind are said to have the same type or, more often, are said to be in the same class.

For example, in an airport simulation program, all the simulated airplanes might belong to the same class, probably called the Airplane class. All objects within a class have the same methods. Thus, in a simulation program, all airplanes have the same methods (or possible actions), such as taking off, flying to a specific location, landing, and so forth. However, all simulated airplanes are not identical. They can have different characteristics, which are indicated in the program by associating different data (that is, some different information) with each particular airplane object. For example, the data associated with an airplane object might be two numbers for its speed and altitude.

Things that are called procedures, methods, functions, or subprograms in other languages are all called methods in Java. In Java, all methods (and for that matter, any programming constructs whatsoever) are part of a class.

## Lab Activities

### Activity 1

The example demonstrates the difference in approach if we want to find the circumference of circle.

### Solution:

| Procedural Approach                                                                                                                                                                                                                                                                                                                                                    | Object Oriented Approach                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                        |
|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <pre> Public class <b>Circle</b>{ int radius; Public void setRadius(int r) { radius = r;} Public void showCircumference() { double c = 2*3.14*radius; <b>System.out.println(" Circumferenceis"+ c);</b> } Public static void main() { setRadius(5); showCircumference(); //output would be 31.4 setRadius(10); showCircumference(); // output would be 62.8 } } </pre> | <pre> Public class <b>Circle</b>{ Private int radius; Public void setRadius(int r) { radius = r;} Public void showCircumference() { double c = 2*3.14* radius; <b>System.out.println(" C i r c u m f e r e n c e   i s"+</b> c); } } Public class <b>runner</b> { Public static void main() { Circle c1= new circle(); c1.setRadius(5); c1.showCircumference(); //output would be 31.4; it belongs to c1 Circle c2= new circle(); c2.setRadius(10); c2.showCircumference(); //output would be 62.8; it belongs to c2 } } </pre> |

### Activity 2

#### Activity 1:

Consider the concept of a CourseResult. The CourseResult should have data members like the student name, course name and grade obtained in that course. This concept can be represented in a class as follows:

### Solution:

```
Public class CourseResult
{
    Public String studentname;
    Public String coursename;
    Public String grade;
    public void display()
    {
        System.out.println("Student Name is: " + studentname + "Course Name is: " + coursename +
            "Grade is: " + grade);
    }
}
```

```
Public class CourseResultRun
{
    public static void main(String[]args)
    {
        CourseResult c1=new CourseResult ();
        c1.studentName= "Ali";
        c1.courseName= "OOP";
        c1.grade= "A";
        c1.display();

        CourseResult c2=new CourseResult ();
        c2.studentName= "Saba";
        c2.courseName= "ICP";
        c2.grade= "A+";
        c2.display();

    }
}
```

Note that both objects; c1 and c2 have three data members, but each object has different values for their data members.

### Activity 3:

The example below represents a Date class. As date is composed of three attributes, namely month, year and day; so the class contains three Data Members. Now every date object will have these three attributes, but each object can have different values for these three

### Solution:

```
public class Date
{
    public String month;
```

```

        public int day;
        public int year; //a four digit number.

        public void displayDate()
        {
            System.out.println(month + " " + day + ", " + year);
        }
    }

    public static void main(String[] args)
    {
        Date date1, date2;
        date1 = new Date();
        date1.month = "December";
        date1.day = 31;
        date1.year = 2012;
        System.out.println("date1:");
        date1.display();
        date2 = new Date();
        date2.month = "July";
        date2.day = 4;
        date2.year = 1776;
        System.out.println("date2:");
        date2.display();
    }
}

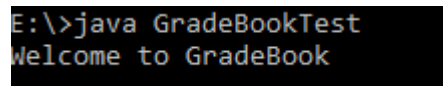
```

## Home Activities

### Activity 1:

Write java code to create the *GradeBook* class that contains a *displayMessage* method to displays a message on the screen. You will need to make an object of this class and call its method to execute display this message “Welcome to GradeBook” Now declare a separate class that contains a main method. The *GradeBookTest* class declaration will contain the main method that will control your application’s execution.

**Expected Output:**



```

E:\>java GradeBookTest
Welcome to GradeBook

```

### Activity 2:

Write a class Circle, which will model the functionality of a Circle.

#### 1. Attributes

☒ radius

#### 2. Methods

☒ *calculateArea()*: To compute area

☒ *calculatePerimeter()*: To compute perimeter

**Note: For value of pi, use Math.PI**

**Expected Output:**

```
Radius is 3.50  
Area is: 38.47  
Perimeter is: 21.98
```

## Assignment

**Task 1:** Write a class Rectangle, which will model the functionality of a Rectangle.

1. Attributes
  - ☒ Length
  - ☒ Width
2. Methods
  - ☒ *calculateArea()*: To compute area
  - ☒ *calculatePerimeter()*: To compute perimeter

**Expected Output:**

```
Length is 4  
Width is 2  
Area is: 8.00  
Perimeter is: 12.00
```

**Task 2:** Write a class Account, which will model the functionality of a bank account.

1. Attributes
  - ☒ AccountTitle
  - ☒ TotalBalance
2. Methods
  - ☒ *deposit(double amountToDeposit)*: To deposit amount to account
  - ☒ *withdraw(double amountToWithdraw)*: To withdraw amount from account. In this method, you have check withdrawal amount should be less than total amount.

### **Statement Purpose:**

Objective of this lab is to understand the importance of classes and construction of objects using constructors and declaration of private instance variables.

### **Activity Outcomes: (CLO4-PLO5/P2)**

The student will be able to declare a classes and objects.

The student will be able to declare PRIVATE member functions and member variables of a class.

The student will be able to declare overloaded constructors.

### **Instructor Note:**

The student should have understanding about object-oriented paradigm.

# **Introduction**

- **Constructors:**

It is a special function that is automatically executed when an object of that class is created. It has no return type and has the same name as that of the class. It is normally defined in classes to initialize data members. A constructor with no parameters is called a no-argument constructor. A constructor may contain arguments which can be used for initiation of data members.

**Syntax:**

```
class_name( )
{
    Public class_name()
    {
        //body
    }

    Public class_name(type var1, type var2)
    {
        //body
    }
}
```

If your class definition contains absolutely no constructor definitions, then Java will automatically create a no-argument constructor. If your class definition contains one or more constructor definitions, then Java does not automatically generate any constructor; in this case, what you define is what you get. Most of the classes you define should include a definition of a no-argument constructor.

## **Encapsulation**

Information hiding means that you separate the description of how to use a class from the implementation details, such as how the class methods are defined. The programmer who uses the class can consider the implementation details as hidden, since he or she does not need to look at them. Information hiding is a way of avoiding information overloading. It keeps the information needed by a programmer using the class within reasonable bounds. Another term for information hiding is abstraction.

Encapsulation means grouping software into a unit in such a way that it is easy to use because there is a well-defined simple interface. So, encapsulation and information hiding are two sides of the same coin.

## **Access Modifiers**

Java allows you to control access to classes, methods, and fields via access modifiers. The access to classes, constructors, methods and fields are regulated using access modifiers i.e. a class can control what information or data can be accessible by other classes. To take advantage of encapsulation, you should minimize access whenever possible.

**Syntax:**

```
class class_name {  
    access_specifier type member1;  
    access_specifier type member1;  
    .....  
}
```

The following table describes the access modifiers provided by JAVA.

| Modifier      | Description                                                     |
|---------------|-----------------------------------------------------------------|
| (no modifier) | member is accessible within its package only                    |
| Public        | member is accessible from any class of any package              |
| Protected     | member is accessible in its class package and by its subclasses |
| Private       | member is accessible only from its class                        |

## **Home Activities:**

### **Activity 1:**

Make a Rectangle class that has color, width and height attribute. Color is of String type, while other two are int type attribute. All the attributes should be private and exposed via setter/getter methods. Value of width and height should be greater than zero.

Define a method inside Rectangle class: *int calculateArea()* that returns area of Rectangle.

Define another method in Rectangle class: *int calculatePerimeter()* that returns perimeter of rectangle. Make a RectangleTest class, in main method, instantiate a rectangle object. Ask user to input value of width and height, and prints its area and perimeter.

### **Activity 2:**

Write a class BankAccount, which have following attributes:

1. Attributes

- ☒ accountNumber
- ☒ accountTitle
- ☒ balance (should be greater than zero)

All attributes are private.

Create a test class named BankAccountTest. In main method, instantiate BankAccount object and display information of account.

**Note: Ask user to input value of accountNumber, accountTitle and balance.**

### Activity 3:

Design a class Cylinder, which has following members:

- ☒ Data:
  - ☒ radius
  - ☒ height
- ☒ Constructor
  - ☒ A no-argument constructor. This constructor print this statement “A no argument constructor”.
  - ☒ A one-argument constructor receiving height and setting radius to default value 1
  - ☒ A two-argument constructor receiving height and radius (demonstrate the use of this reference by using same names for parameters and instance variables)
- ☒ Methods:
  - ☒ Setter functions (total 2)
  - ☒ Getter functions (total 2)
  - ☒ computeArea
  - ☒ computeVolume
    - ☒ look for the relevant formulae over internet
  - ☒ displayInfo
    - ☒ Showing cylinder height, radius, area and volume
- ☒ Test class to test Cylinder
  - ☒ Demonstrate all the above constructors/methods by creating different objects and calling each constructor/method at least once

## Home Assignment

Write a date class to model a date object.

‣ Date

- year
- month
- day

Value of day should not be greater than 30, if day value is greater than 30 reset it to one.

Value of month should not be greater than 12, if month value is greater than 12 then reset it to one.

☒ Constructors

- ☒ No argument constructor. In this constructor set Day value to 7, month value to 10 and year to 2020.
- ☒ One parameterized constructor for setting all three fields

☒ Methods

- ☒ A method to print date in “dd/mm/yy” format
- ☒ Implement the setter and getter methods for day and month.

☒ Write DateTest class for the Date class

- ☒ Create two date objects **d1** and **d2** by calling constructors with appropriate values
- ☒ Input date, month and year from user using Scanner object
- ☒ Create two reference variables (**d3** and **d4**) of type date.
- ☒ Print date for **d1**, **d2** by calling print method of date.
- ☒ Assign **d2** to a new object **d3** and **d3** to **d4**
- ☒ Change **d3** month through the setter methods

Print **d1**, **d2**, **d3** and **d4** by calling print method of date and observe the effect of reference types

### **Statement Purpose:**

This Lab will help students in understanding the concept static data and member function of a class.

### **Activity Outcomes: (CLO4-PL05/P3)**

After completion of this Lab students know in which scenario static data members, static functions and static objects can be used.

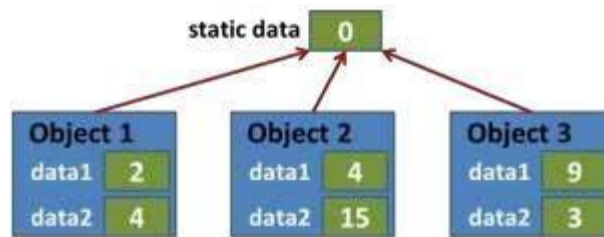
### **Instructor Note:**

The student should have understanding about access specifiers.

The students should understand **the concept of object's address space in memory.**

## Introduction

There is an important exception to the rule that each object of a class has its own copy of all the data members of the class. In certain cases, only one copy of a variable should be shared by all objects of a class. A **static data member** is used for such propose.. Such a variable represents "class-wide" information (i.e., a property of the class shared by all instances, not a property of a specific object of the class). The declaration of a static member begins with keyword static.



In the above figure, consider Object1, Object2 and Object3 are three of a class. Each object has its own private data members. The class has a static data member that is share among all the object of that class. This data member has it own memory allocated that is separate from the memory allocated for each object of that class.

### Static data member – access

A public static data member can be access using any object of the class or class name

- [Object Name].[name of static variable]
  - e1.count;
- [Class Name].[name of static variable]
  - Employee .count;

A private static member can be access using a public member function

- [objectName].[public member function]
  - e1.getCount();
- [Class name].[public member function]
  - Employee.getCount();

A static method is a method that can only access static data member of class. It cannot access non static data member of that class

### Static vs. non static functions

- Static function can only access static data member of class
  - className.static\_ method ();
  - Object\_of\_class.static\_method();
- Non static member function can access static and non static data member of class
  - Object\_of\_class.Non\_static\_ method ();

## **Lab Activities:**

### **Activity 1:**

The following example demonstrates that static variables are common for all instances:

### **Solution:**

```
Public class ABC
{
    static int Var1=77;           //Static integer variable
    String Var2;                 //non-static string variable
}
Public class ABCRunner
{
    public static void main(String args[])
    {
        ABC ob1 = new ABC ();
        ABC ob2 = new ABC ();
        ob1.Var1=88;
        ob1.Var2="I'm Object1";
        ob2.Var2="I'm Object2";
        System.out.println("ob1 integer:"+ob1.Var1);
        System.out.println("ob1 String:"+ob1.Var2);
        System.out.println("ob2 integer:"+ob2.Var1);
        System.out.println("ob2 String:"+ob2.Var2);
    }
}
```

### **Output:**

```
ob1 integer:88
ob1 String:I'm Object1
ob2 integer:88
ob2 String:I'm Object2
```

### **Activity 2:**

The following example demonstrates counting of objects by using static variable.

### **Solution:**

```
Public class NoOfObjects {
    Private static int objs=0;
```

```

    Private int a;

    Public NoOfObjects(){
        objs++;
    }

    Public NoOfObjects(intx){
        a=x;
        objs++;
    }

    Public static int getObjs (){
        return objs;
    }
}
Public class NoOfObjectsRunner
{

    Public static void main(String[] args){

        NoOfObjects o1=newNoOfObjects();
        NoOfObjects o2=newNoOfObjects(122);
        NoOfObjects o3=newNoOfObjects(150);

        System.out.println("Objects created:"+
        NoOfObjects.getObjsCreated()); System.out.println("Objects
        created:"+ o1.getObjsCreated()); }
}

```

### Activity 3:

The following example demonstrates static methods declaration.

```

Public class ABC
{
    Public static int i ;
    public String s;

    Public static void displayStatic() //Static method
    {
        System.out.println("i:"+i);
    }
    Public void display()           //non static method
    {
        System.out.println("i:"+i);
        System.out.println("s:"+s);
    }
}

```

```

Public class ABCRunner
{
    public static void main(String args[]) //Its a Static Method
    {
        ABC a = new ABC();
        a.display();
        ABC.displayStatic();
    } }

```

## **Home Activities:**

### **Activity 1:**

Create a SavingsAccount class. Use a static data member annualInterestRate to store the annual interest rate for each of the savers. Each member of the class contains a private data member savingsBalance indicating the amount the saver currently has on deposit. Provide member function calculateMonthlyInterest that calculates the monthly interest by multiplying the balance by annualInterestRate divided by 12; this interest should be added to savingsBalance.

Provide a static member function modifyInterestRate that sets the static annualInterestRate to a new value.

Write a driver program to test class SavingsAccount. Instantiate two different objects of class SavingsAccount, saver1 and saver2, with balances of \$2000.00 and \$3000.00, respectively. Set the annualInterestRate to 3 percent. Then calculate the monthly interest and print the new balances for each of the savers.

Then set the annualInterestRate to 4 percent, calculate the next month's interest and print the new balances for each of the savers.

## **Assignment:**

Write a JAVA program that creates a class GuessTheWord, where class attributes is a static variable score(int).

In GuessTheWord create a static method, levelOne(with 3 string type arguments). In this method, ask user to input the word, and then you have compare it with words given in method arguments.

If user input = word 1, score will have -1; if user input = word 2, score will have +5 and if user input = word3, score will have +1.

Now make another class `GuessTheWordLevelTwo` which will have another static method `levelTwo` which will give bonus of 10 and print the score and also print “You are Now at LEVEL 2”

In main (`GameTest` Class) `levelTwo` will be loaded only when score value will be greater than or equal to 10.

### **Statement Purpose:**

The purpose of lab is to make students understand the concept of has-a relationship in object oriented programming.

### **Activity Outcomes: (CLO5-PL010/P1)**

Students will be able to understand that complex objects can be modeled as composition of other objects.

Students will be able to implement programs related to composition.

### **Instructor Note:**

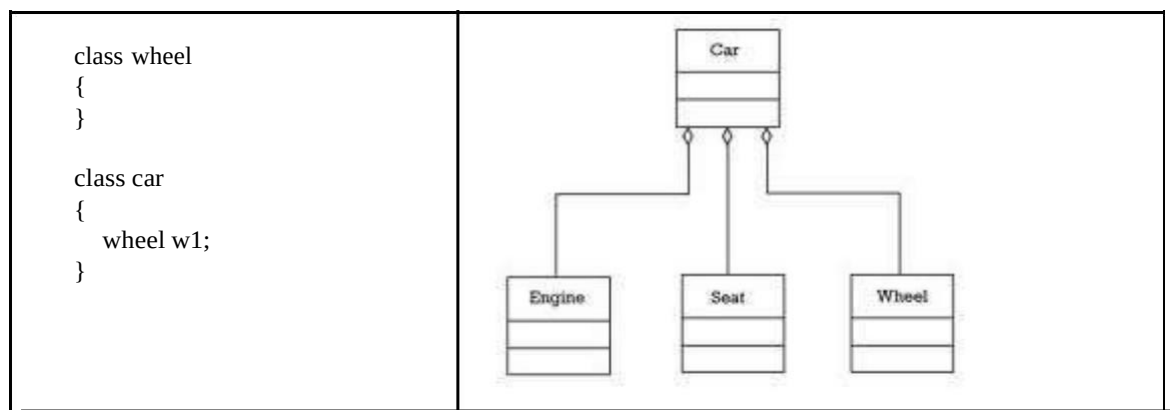
The student should be able to declare a class with primitive data members.

The student should be able to define methods; that can receive and return objects.

## Introduction

In real-life, complex objects are often built from smaller, simpler objects. For example, a car is built using a metal frame, an engine, some tires, a transmission, a steering wheel, and a large number of other parts. A personal computer is built from a CPU, a motherboard, some memory, etc. Even you are built from smaller parts: you have a head, a body, some legs, arms, and so on. This process of building complex objects from simpler ones is called composition (also known as object containership).

More specifically, composition is used for objects **that have a “has-a” relationship to each** other. A car has-a metal frame, has-an engine, and has-a transmission. A personal computer has-a CPU, a motherboard, and other components. You have-a head, a body.



## Lab Activities:

### Activity 1:

Composition is about expressing relationships between objects. Think about the example of a manager. A manager has the properties e.g Title and club dues. A manager has an employment record. And a manager has an educational record. The phrase "has a" implies a relationship where the manager owns, or at minimum, uses, another object. It is this "has a" relationship which is the basis for composition. This example can be programmed as follows:

### Solution:

```
public class studentRecord
{
    Private String degree;

    public studentRecord()
    {}
}
```

```

        public void setDegree ( String deg)
        {
            degree = deg;
        }

        public void getDegree ()
        {
            return degree;
        }
    }

    class employeeRecord
    {
        Private int emp_id;
        Private double salary;
        public employeeRecord ()
        {}
        public void setEmp_id ( int id)
        {
            emp_id = id;
        }
        public intgetEmp_id ()
        {
            return emp_id ;
        }
        public void setSalary ( intsal)
        {
            Salary = sal;
        }
        public StringgetSalary ( intsal)
        {
            Salary = sal;
        }
    }

    class Manager
    {
        Private String title;
        Private double dues;
        Private employeeRecordemp;
        Private studentRecordstu;

        Public manager(String t,double d,employeeRecord e,studentRecord s)
    }

```

```

{
    titile = t;
    dues = d;
    emp = e;
    stu = s; }

Public void display()
{
    System.out.println("Title is : " + title);
    System.out.println("Dues are : " + dues);

    System.out.println("Emplyoyee record is as under:" );
    System.out.println("EmployeeId is : " + emp. getEmp_id());
    System.out.println("EmployeeId is : " + emp. getSalary());

    System.out.println("Student record is as
under:" ); System.out.println("Degree is : "
+ stu.getDegree()); }

}
Public class Runner
{
    voidmain()
    {
        studentRecord s = new studentRecord("MBA");
        employeeRecord e = new employeeRecord(111, 50000);
        Manager m1 = new Manager("financeManager ", 5000, e, s );
        m1.display();
    }

}

```

## Activity 2:

The program below represents an employee class which has two Date objects as data members.

## Solution:

```

public class Date

{

    private int day;
    private int month;
    private int year;

```

```

public Date (int theMonth, int theDay, int theYear )
{
    day=checkday(theDay);
    month=checkmonth(theMonth);
    year=theYear;
}

Private intcheckmonth (int testMonth)
{
    if (testMonth>0 &&testMonth<=12)
        returntestMonth;

else
{
    System.out.println("Invalid month" + testMonth + "set to
1"); return 1;
}
}
Private int checkday(int testDay)
{
    intdaysofmonth[]={0,31,28,31,30,31,30,31,31,30,31,30,31};

    if (testDay>0 &&testDay<=daysofmonth[month])
        return testDay;

else
    if (month==2 &&testDay==29 && (year%400==0 || (year%4==0 &&
year%100 !=0)))
        return testDay;

else
    System.out.println("Invalid date"+ testDay + "set to 1");
    return 1;
}
Public int getDay()
{
    return day;
}

Public int getMonth()
{
    return month;
}

Public int getYear()
{

```

```

        return year;
    }

    public void display()
    {
        System.out.println(day + " " + month + " " + year);
    }

}

public class employee
{
    private String name;
    private String fname;
    private Date birthdate;
    private Date hiredate;
    else
    {
        System.out.println("Invalid month" + testMonth + "set to
1"); return 1;
    }
}
Private int checkday(int testDay)
{
    int daysofmonth[]={0,31,28,31,30,31,30,31,31,30,31,30,31};

    if (testDay>0 &&testDay<=daysofmonth[month])
        return testDay;

    else
        if (month==2 &&testDay==29 && (year%400==0 || (year%4==0 &&
year%100 !=0)))
            return testDay;

    else
        System.out.println("Invalid date"+ testDay + "set to 1");
        return 1;
}
Public int getDay()
{
    return day;
}

Public int getMonth()
{
    return month;
}

```

```

    }

    public int getYear()
    {
        return year;
    }

    public void display()
    {
        System.out.println(day +" "+ month+" " + year);
    }

}

public class employee
{
    private String name;
    private String fname;
    private Date birthdate;
    private Date hiredate;
    public void display()
    {

        System.out.println("Name: "+ name +" Father Name: "+ fname );
        birthdate.display();
        hiredate.display();

    }
}

public class Employrun {

    public static void main(String[] args)
    {
        Date b = new Date (1,12,1990);
        Date h = new Date (5,6,2016);
        employee e1=new employe("xxx", "yyyy",b,h);
        e1.display();
    }

}

```

## **Home Activities:**

### **Activity 1:**

Make a class **Address** that have three attributes, streetNumber, city and country. All attributes are private and will be exposed using getter and setter method.

Make an **Employee** class, which have employeeID, employeeName and employeeAddress. All attributes are private. Attributes employeeAddress should be of type Address. In this class, define a constructor, which takes two parameters i.e. employeeID and employeeAddress.

Make another class **EmployeeTest**, in main method, instantiate an employee object named employee1. Initialize id and address attribute using constructor. The attribute name should be initialized using setter method. User should give value of name attribute. Also, print values of all attributes using getter.

Make another employee object named employee2. Both employees share same address. Print the id and address for employee2.

### **Activity 2:**

Make a class **Date** with three attributes, day, month and year. All attribute are private. In constructor, you have to validate day. If day is out of range, you have to print a message, "Invalid Date". Suppose all months have 30 days.

Make a class **Employee** that has four instance variable firstName, lastName, birthdate and hiringDate. firstName and lastName are reference to String object while birthdate and hiringDate are references to Date object. All instance variable are private. Employee class has a fully parameterized constructor.

Make a class **Date** with three attributes, day, month and year. All attribute are private. In constructor, you have to validate day. If day is out of range, you have to print a message, "Invalid Date". Suppose all months have 30 days.

Make a class **Employee** that has four instance variable firstName, lastName, birthdate and hiringDate. firstName and lastName are reference to String object while birthdate and hiringDate are references to Date object. All instance variable are private. Employee class has a fully parameterized constructor.

## **Assignment:**

For each class/attributes described below, choose appropriate data type. All attributes of each class should be private and exposed via get/set methods. Also define at least one constructor shall take and initialize 2-3 attributes of the object.

1. Define a class **Course** with courseCode and courseTitle attributes.
2. Define a class **PhoneNumber** with countryCode, cityCode and lineNumber attributes.

3. Define a class **Address** with streetAddress, town, city, country and phoneNumber attributes. Attribute phoneNumber shall be of type PhoneNumber.
4. Define a class **Student** with name, email, cnic, course1, course2, postalAddress and contactNumber attributes. Where course1 and course2 should be of type Course, postalAddress shall be of type Address; contactNumber should be of Type PhoneNumber. Define a constructor in Student class that takes cnic, name and address only.

Create a **StudentTest** class, in its main method, create a Student object i.e. student1. Fully initialize its' all attributes. cnic, name and address shall be initialized by constructor, other attributes shall be initialized using setter methods. **All attributes values shall be taken from user.** After the object is fully initialized, print all, attribute values from student object reference.

Make another object student2, assume the student live at same address as student1. Reuse the address object of student1 to initialize student2 address. You do not need to take attributes from user input for student2 object. Change some attribute of address from student1 and check, does it also change for student2, understand why and why not?

### **Statement Purpose:**

The purpose of lab is to make students understand the concept of primitive and reference arrays in java.

### **Activity Outcomes: (CLO5-PL010/P1)**

Students will be able to understand that the difference primitive and reference arrays.  
Students will be able to implement one dimensional as well as multi-dimensional arrays in java.

### **Instructor Note:**

The Students should have knowledge about arrays data structure.

The student should be able to define primitive arrays.

## **Introduction**

One-D array only held one column of data. However, you can set up an array to hold more than one column. These are called multi-dimensional arrays.

Syntax is very simple:

```
int nums[][] = new int[3][4];
```

We have declared and constructed the array

MD arrays are array of arrays. So 3 indicates the 'nums' array is of size 3 (3 rows). And 4 indicates, each element points to any array of size 4 (4 columns)

nums length is 3, not 12 (you must understand, why?)

nums[0] length is 4

## **Lab Activities:**

### **Activity 1:**

The program below represents an example of multidimensional array in java.

### **Solution:**

```
public static void main(String[] args) {  
  
    int[][] aryNumbers = new int[6][5];  
  
    aryNumbers[0][0] = 10;    aryNumbers[1][0] = 20;  
    aryNumbers[0][1] = 12;    aryNumbers[1][1] = 45;  
    aryNumbers[0][2] = 43;    aryNumbers[1][2] = 56;  
    aryNumbers[0][3] = 11;    aryNumbers[1][3] = 1;  
    aryNumbers[0][4] = 22;    aryNumbers[1][4] = 33;  
  
    aryNumbers[2][0] = 30;    aryNumbers[3][0] = 40;  
    aryNumbers[2][1] = 67;    aryNumbers[3][1] = 12;  
    aryNumbers[2][2] = 32;    aryNumbers[3][2] = 87;  
    aryNumbers[2][3] = 14;    aryNumbers[3][3] = 14;  
    aryNumbers[2][4] = 44;    aryNumbers[3][4] = 55;  
}
```

```

aryNumbers[4][0] = 50;      aryNumbers[5][0] = 60;
aryNumbers[4][1] = 86;      aryNumbers[5][1] = 53;
aryNumbers[4][2] = 66;      aryNumbers[5][2] = 44;
aryNumbers[4][3] = 13;      aryNumbers[5][3] = 12;
aryNumbers[4][4] = 66;      aryNumbers[5][4] = 11;

int rows = 6;
int columns = 5;
int i, j;

for ( i = 0; i < rows; i++) {
    for ( j = 0; j < columns; j++) {
        System.out.print(aryNumbers[i][j] + " ");
    }
    System.out.println( "" );
}

```

## Activity 2:

Another program of multidimensional array in java.

## Solution:

```

package mdarrayclassexample2;

import java.util.Scanner;

/**
 *
 * @author AC
 */
public class MDArrayClassExample2 {

    /**
     * @param args the command line arguments
     */
    public static void main(String[] args) {
        // TODO code application logic here
        Scanner sc=new Scanner(System.in);
        System.out.print("Enter size of array:");
        int size=sc.nextInt();
        int arr[][]=new int[size][];
        for(int i=0;i<arr.length;i++)
        {

```

```

        System.out.print("Enter size of array "+(i+1)+":");
        arr[i]=new int[sc.nextInt()];
        System.out.print("Enter elemets of array:");
        for(int j=0;j<arr[i].length;j++)
        {
            arr[i][j]=sc.nextInt();
        }
    }
    for(int a[:arr)
    {
        for(int b:a)
        {
            System.out.print(b+" ");
        }
        System.out.println();
    }
}
}

```

## **Home Activities:**

### **Activity 1:**

Make an array of n elements and initialize it, data and size of the array shall be given by user. Print all elements, print minimum number and its index in array, print maximum number and its index in the array and also print the average number.

### **Activity 2:**

Make a class Course with following attributes courseCode, courseTitle and courseCreditHr. All attributes are private and exposed with getter and setter. In main class, create an array of courses. Ask user to input number of courses and then for each course, get all data from user. Once user input is done, print information of courses in following format:

#### **Course 1 Data:**

Course Title: Object Oriented Programming  
 Course Code: CSC241  
 Course Credit Hour: 4

### **Activity 3:**

Extend Task 1 (from Lab 6), such that a Student may have more than one address i.e. you shall define addresses attribute in Student class that shall be an array of Address. In main

method, create an array of Student that represent all students in a classroom. Let user input the number of students and student's attributes (id and name shall be enough). Let user enter the number of addresses of each student need to store. Get address attributes from user input too. Print all the information on console after the objects are initialized.

### **Assignment:**

Make a class Course with attributes courseTitle and 2-dimensional int array of marks.  
i.e.

```
int[][] marks=new int[noOfStudents][];
```

First user will input total number of students and then number of test for each student.

Create a method inputMarksData() which will input marks of all student in a course. Create another method displayStudentData() which will print courseTitle and marks of each student. In main class instantiate object of Course class, ask user to input courseTitle and call both methods.

### **Statement Purpose:**

The purpose of lab is to make students understand the concept of array list data structure in java.

### **Activity Outcomes: (CLO5-PL010/P1, P2)**

Students will be able to use effectively array list data structure.

### **Instructor Note:**

The Students should have knowledge about arrays data structure and basic operations and methods of array data structure.

## **Introduction**

The Java ArrayList is a dynamic array-like data structure that can grow or shrink in size during the execution of a program as elements are added/deleted. An Array on the other hand, has a fixed size: once we declared it to be a particular size, that size cannot be changed. To use an ArrayList, you first have to import the class:

```
import java.util.ArrayList;
```

You can then create a new ArrayList object:

```
ArrayList<T> Test = new ArrayList<>( );
```

The Java API has a list of all the methods provided by an ArrayList.

## **Lab Activities:**

### **Activity 1:**

The following program shows a simple use of ArrayList. An array list is created, and then objects of type String are added to it. The list is then displayed. Some of the elements are removed and the list is displayed again.

### **Solution:**

```
package arraylistclassexample1;

import java.util.ArrayList;
public class ArrayListClassExample1 {
    public static void main(String[] args) {
        // TODO code application logic here
        ArrayList<String> list=new ArrayList<>();
        //list.
        list.add("Tooba");
        list.add("Maryam");
        list.add("Maryam");
        list.add("Tooba");
        System.out.println("Total size of list:"+list.size());
        display(list);
        System.out.println(list.lastIndexOf("Tooba"));

        list.add(0,"Ali");
        System.out.println("Total size of list:"+list.size());
    }
}
```

```

        display(list);

        list.remove("Tooba");
        System.out.println("Total size of list:"+list.size());
        display(list);

        list.remove(1);
        System.out.println("Total size of list:"+list.size());
        display(list);

        list.set(0, "Ayesha");
        System.out.println("Total size of list:"+list.size());
        display(list);

        list.clear();
        System.out.println("Total size of list:"+list.size());
        display(list);
    }
    public static void display(ArrayList<String> a)
    {
        for(String x:a)
        {
            System.out.println(x);
        }
        System.out.println();
    }
}

```

## **Home Activities:**

### **Activity 1:**

- Create an arraylist of strings named student and apply given methods on array list.
  - Add name of students in arrayList
    - Ali
    - Ahmad
    - Umar
    - Talha
- Create a static function named *display(ArrayList<String> students)* and call it after each of following steps and also print a blank line after each step.
- Add two more students at the end of students list using students.add()
  - 1)Tooba 2) Waleed

- Print the size of list.
- Add “Hamza” in the start of students.
- Add “Rizwan” as second element of list.
- Sort list of students
- Remove “Tooba” from list.
- Remove last element from list.

Hint: To remove last element use (size()-1)

- Display second element from list.
- Update name of first student. New name should be “Muhammad Waleed”
- Remove the first name.
- Remove all elements from list.

## **Assignment:**

Create a fully encapsulated class named Student.

Student class has three attributes name, id and CGPA. Create a constructor, which take two attributes i.e. id and name.

- Create test class with name StudentTest

### **In main class:**

- Create three objects of Student named s1, s2 and s3 with following attributes
  - Ali, SP19-BSE-010
  - Ahmad", SP19-BSE-001
  - Talha, SP19-BSE-002
- Create a static function named *display*, call it after each of following steps, and print a blank line after each step. This method should display id, name and CGPA of student.

Hint: set CGPA using setCGPA(double cgpa) method

- Create an ArrayList named arraylist and add the above objects to arraylist
- Add another Student object at the start of ArrayList with the following attributes
  - Waleed, SP19-BSE-020
- Add another Student object at the end of ArrayList with the following attributes
  - Maha, SP19-BSE-002
- Remove second student from list using index.
- Remove first student from list using object reference.
- Print total number of students.
- Replace the student at index 0 with new student.
- Now display information of first student in list.

### **Statement Purpose:**

The objective of this lab is to familiarize the students with various concepts and terminologies of Java specially Inheritance, Method Overriding and Polymorphism

### **Activity Outcomes: (CLO5-PL010/P2, P3)**

Students will be able to understand that how new classes are created with copying the code of existing classes.

This lab teaches you method overriding where a base class method version is redefined in the child class with exact method signatures.

The student will understand the basic paradigm of OOP i.e. Polymorphism.

### **Instructor Note:**

The Students should have knowledge about classes objects & methods in Java.

## **Introduction**

### **a. Inheritance**

Inheritance is a way of creating a new class by starting with an existing class and adding new members. The new class can replace or extend the functionality of the existing class. The existing class is called the base class and the new class is called the derived class.

### **b. Protected Access Specifier**

Protected members are directly accessible by derived classes but not by other users. A class member labeled **protected** is accessible to member functions of derived classes as well as to member functions of the same class.

### **c. Derived class constructor**

Constructors are not inherited, even though they have public visibility. However, the super reference can be used within the child's constructor to call the parent's constructor. In that case, the call to parent's constructor must be the first statement.

### **d. Method Overriding**

The definition of an inherited method can be changed in the definition of a derived class so that it has a meaning in the derived class that is different from what it is in the base class. This is called overriding the definition of the inherited method.

For example, the methods `toString` and `equals` are overridden (redefined) in the definition of the derived class `HourlyEmployee`. They are also overridden in the class `SalariedEmployee`. To override a method definition, simply give the new definition of the method in the class definition, just as you would with a method that is added in the derived class.

In a derived class, you can override (change) the definition of a method from the base class. As a general rule, when overriding a method definition, you may not change the type returned by the method, and you may not change a void method to a method that returns a value, nor a method that returns a value to a void method. The one exception to this rule is if the returned type is a class type, then you may change the returned type to that of any descendent class of the returned type. For example, if a function returns the type `Employee`, when you override the function definition in a derived class, you may change the returned type to `HourlyEmployee`, `SalariedEmployee`, or any other descendent class of the class `Employee`. This sort of changed return type is known as a covariant return type and is new in Java version 5.0; it was not allowed in earlier versions of Java.

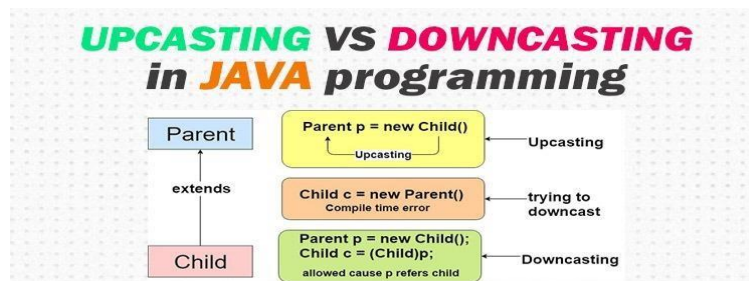
Generally, the ability to appear in many forms. In object-oriented programming, *polymorphism* refers to a programming language's ability to process objects differently depending on their data type or class. More specifically, it is the ability to redefine *methods* for *derived classes*. For example, given a base class *shape*, polymorphism enables the programmer to define different *area* methods for any number of derived classes, such as circles, rectangles and triangles. No matter what shape an object is, applying the *area* method to it will return the correct results. Polymorphism is considered to be a requirement of any true object-oriented programming language (OOPL).

Polymorphism could be static and dynamic both. Method Overloading is static polymorphism while, Method overriding is dynamic polymorphism.

- **Overloading** in simple words means more than one method having the same method name that behaves differently based on the arguments passed while calling the method. This is called static because, which method to be invoked is decided at the time of compilation
- **Overriding** means a derived class is implementing a method of its super class. The call to overridden method is resolved at runtime, thus called runtime polymorphism

#### Up casting and down casting:

- ♦ You cannot automatically make reference variable of subclass type point to object of its superclass
- ♦ Suppose that supRef is a reference variable of a superclass type and supRef points to an object of its subclass:
  - ♦ Can use a cast operator on supRef and make a reference variable of the subclass point to the object
  - ♦ If supRef does not point to a subclass object and you use a cast operator on supRef to make a reference variable of the subclass point to the object, then Java will throw a ClassCastException—indicating that the class cast is not allowed



- ♦ Operator instanceof: determines whether a reference variable that points to an object is of a particular class type
- ♦ This expression evaluates to true if p points to an object of the class BoxShape; otherwise it evaluates to false: p instanceof BoxShape

## **Lab Activities:**

This example will explain the method to specify the derived class. It explains the syntax for writing the constructor of derived class.

### **Solution:**

```
public class person {

    protected String name ;
    protected String id ;
    protected int phone ;

    public person() {
        name = "NaginaNazar" ;
        id = "sp14bcs039" ;
        phone = 12345 ;
    }

    public person(String a , String b , int c) {
        name = a ;
        id = b ;
        phone = c ;
    }

    public void setName(String a){
        name = a ;
    }

    public void setId(String j){
        id = j ;
    }

    public void setPhone(int a) {
        phone = a ;
    }

    public String getName() {
```

```

        return name ;
    }

    public String getId() {
        return id ;
    }

    public int getPhone() {
        return phone ;
    }
}

public class student extends person {
    private String rollNo ;
    private int marks ;

    public student() {
        super() ;
        rollNo = "sp14bcs039" ;
        marks = 345 ;
    }

    public student(String a , String b , int c
        , String d , int e){ super(a,b,c) ;
        rollNo = d ;
        marks = e ;
    }

    public void setRollNo(String a){
        rollNo = a ;
    }

    public void setMarks(int a ){
        marks = a ;
    }

    public String getRollNo() {
        return rollNo ;
    }

    public int getMarks() {
        return marks ;
    }

    public void display(
    ) {
        System.out.println("Name : " + name + "ID : " + id + "Phone : " + phone );
    }
}

```